Lecture Supervision on a Circular Timetable: Efficient Open Domination of a Circular-Arc Graph

Design and Analysis of Algorithms – Course Project

October 22, 2026

Contents

1 Problem statement and classification **1**

2 Structure of every solution **1**

3 Algorithm **2**

4 Implementation and test run **3**

5 Correctness **4**

6 Complexity **4**

7 Live test cases (executed while this document compiles) **4**

8 Implementation (README) **5**

1 Problem statement and classification

There are n lectures repeating every day. Lecture i is a circular arc $[s_i, e_i)$ of a circle of length $L = 1440$ minutes and may wrap past midnight. Two lectures *overlap* if they share positive length (touching endpoints do not overlap). Find $S \subseteq \{1, \dots, n\}$ such that every lecture overlaps *exactly one* lecture of S (a lecture never counts itself).

Classification. Let G be the overlap graph (a *circular-arc graph*) and $\mathrm{N}(v)$ the open neighbourhood of v. We need S with $|\mathrm{N}(v) \cap S| = 1$ for all v: an *efficient open dominating set* (total perfect code). Technique: structural characterisation + dynamic programming on a circular sequence (linearised by anchoring), with bit-set graph operations. Exhaustive search would take $O(2^n)$; the algorithm below takes $O(n^3)$.

2 Structure of every solution

Definition 1 (Block)**.** A pair $\{a, b\}$ of overlapping lectures with $\mathrm{N}(a) \cap \mathrm{N}(b) = \emptyset$. Its region $R = a \cup b$ is a union of two overlapping arcs.

Lemma 1 (Decomposition)**.** *S is valid iff S is the union of blocks $B_1, \dots, B_k$ ($k \geq 1$) with regions $R_1, \dots, R_k$ such that*

(C1) the regions are pairwise disjoint;

(C2) no lecture overlaps two different regions;

(C3) every lecture overlaps some region.

Proof. ($\Rightarrow$) Each $v \in S$ has exactly one neighbour in S, so S induces a perfect matching; its edges are blocks (a common neighbour of a, b would see two chosen lectures). Lectures from different pairs do not overlap, giving (C1). A lecture touching two regions would overlap two chosen lectures (it cannot be chosen, since chosen lectures touch only their partner), giving (C2). Every lecture needs a chosen neighbour, giving (C3).

($\Leftarrow$) A chosen lecture $a \in B_j$ overlaps its partner and, by (C1), nothing else chosen. A non-chosen w overlaps some region by (C3), at most one by (C2), and inside it exactly one of a, b because $\mathrm{N}(a) \cap \mathrm{N}(b) = \emptyset$. □

Lemma 2 (Few blocks)**.** *There are at most $2n$ blocks whose region is not the whole circle.*

Proof. If $b \subseteq a$, every lecture touching b touches a, so $\mathrm{N}(b) \setminus \{a\} \subseteq \mathrm{N}(a) \cap \mathrm{N}(b) = \emptyset$: b's only neighbour is a. Hence b determines a: at most n nested blocks. If a, b cross (not nested) the overlap region lies in the tail or head of a and no third lecture may meet it. Two different partners b_1, b_2 at the tail of a would force one to meet the other's overlap region with a, a contradiction. So a has at most one tail partner and one head partner; counting each crossing block once at the lecture whose *tail* it uses gives at most n crossing blocks. □

3 Algorithm

Idea. Lecture 1 overlaps exactly one block in any solution. Try each block that overlaps lecture 1 as an *anchor*; place it at coordinate 0, unroll the circle to $[0, L]$ and decide by a left-to-right DP whether the remaining blocks can be chained so that (C1)–(C3) hold.

For anchor-relative coordinates let, for a position p,

$$Q(p) = \max\{\text{end of a lecture copy starting before } p\},$$
$$T(p) = \min\{\text{end of a lecture starting at or after } p\}.$$

(Copies shifted by $-L$ account for lectures crossing the anchor's start.) A block X ending at x_e may be followed by a block Y starting at y_s iff

$$x_e \le y_s, \qquad Q(x_e) \le y_s \text{ (no lecture meets both)}, \qquad y_s < T(x_e) \text{ (no lecture lies strictly in the gap)}.$$

Closing the cycle is the same test with $y_s = L$; with $k = 1$ only the gap test is needed.

Algorithm 1 SelectLectures(arcs)

build bit-masks of covered minutes and adjacency bit-sets $\mathrm{N}(\cdot)$
$\mathcal{B} \leftarrow \{\{a, b\} : a \cap b \neq \emptyset,\ \mathrm{N}(a) \cap \mathrm{N}(b) = \emptyset\}$
if some block of $\mathcal{B}$ covers the whole circle **then return** it
for each $X_0 \in \mathcal{B}$ whose region meets lecture 1 **do**
    shift coordinates so $X_0 = [0, \ell_0)$; compute Q, T by prefix max / suffix min
    $\mathcal{C} \leftarrow$ blocks lying in $[\ell_0, L]$, sorted by start
    if $L < T(\ell_0)$ **then return** X_0 ▷ $k = 1$
    for $Y \in \mathcal{C}$ in order of start **do**
        $reach[Y] \leftarrow \exists X \in \{X_0\} \cup \mathcal{C}$ reachable before Y with $x_e \le y_s,\ Q(x_e) \le y_s < T(x_e)$
        if $reach[Y]$ **and** $Q(y_e) \le L < T(y_e)$ **then return** chain to Y
return None

4 Implementation and test run

Source code. The complete Python implementation, the sample input and the README are available at:

`https://github.com/YOUR-USERNAME/YOUR-REPOSITORY`

The repository contains `lecture_solver.py` (algorithm, independent verifier, brute-force checker), `sample.txt` and `README.md`. To run your own data: `python3 lecture_solver.py mydata.txt`, where the file holds n followed by n lines `HH:MM HH:MM` (start, end).

Test cases executed during compilation. The eight built-in test cases and the sample file are run now, while this PDF is being built (`pdflatex -shell-escape`); the text below is the genuine program output, not typed by hand. "Independent verification" re-checks, from scratch, that every lecture overlaps exactly one selected lecture.

```
--- T1  two overlapping lectures (n = 2) ---
Selected lectures (1-indexed): [1, 2]
  lecture   1 09:00-10:00  supervised by [2]
  lecture   2 09:30-10:30  supervised by [1]
Independent verification: PASSED
--- T2  night lecture wraps midnight (n = 2) ---
Selected lectures (1-indexed): [1, 2]
  lecture   1 23:00-01:00  supervised by [2]
  lecture   2 00:30-02:00  supervised by [1]
Independent verification: PASSED
--- T3  two chains around the day (n = 4) ---
Selected lectures (1-indexed): [1, 2, 3, 4]
  lecture   1 01:00-03:00  supervised by [2]
  lecture   2 02:00-04:00  supervised by [1]
  lecture   3 13:00-15:00  supervised by [4]
  lecture   4 14:00-16:00  supervised by [3]
Independent verification: PASSED
--- T4  four blocks, private lectures (n = 8) ---
Selected lectures (1-indexed): [1, 2, 4, 6, 7, 8]
  lecture   1 00:00-02:00  supervised by [2]
  lecture   2 01:00-03:00  supervised by [1]
  lecture   3 00:30-01:00  supervised by [1]
  lecture   4 08:00-10:00  supervised by [6]
  lecture   5 09:00-11:00  supervised by [4]
  lecture   6 08:30-09:00  supervised by [4]
  lecture   7 16:00-18:00  supervised by [8]
  lecture   8 17:00-19:00  supervised by [7]
Independent verification: PASSED
--- T5  lecture touched by two blocks (n = 5) ---
No valid subset exists.
--- T6  full-day lecture + private partner (n = 2) ---
Selected lectures (1-indexed): [1, 2]
  lecture   1 00:00-00:00  supervised by [2]
  lecture   2 10:00-11:00  supervised by [1]
Independent verification: PASSED
--- T7  isolated lecture (n = 3) ---
No valid subset exists.
--- T8  touching endpoints do not overlap (n = 2) ---
No valid subset exists.
```

Run on `sample.txt`.

```
--- sample.txt (n = 8) ---
Selected lectures (1-indexed): [1, 2, 4, 6, 7, 8]
  lecture   1 00:00-02:00  supervised by [2]
  lecture   2 01:00-03:00  supervised by [1]
  lecture   3 00:30-01:00  supervised by [1]
```

```
  lecture   4 08:00-10:00  supervised by [6]
  lecture   5 09:00-11:00  supervised by [4]
  lecture   6 08:30-09:00  supervised by [4]
  lecture   7 16:00-18:00  supervised by [8]
  lecture   8 17:00-19:00  supervised by [7]
Independent verification: PASSED
```

5 Correctness

Theorem 1. SELECTLECTURES *returns a valid subset if one exists and* NONE *otherwise.*

Proof. Soundness. A returned chain consists of blocks, so each region has $\mathrm{N}(a) \cap \mathrm{N}(b) = \emptyset$. Regions are disjoint by the condition $x_e \le y_s$ and by restricting candidates to $[\ell_0, L]$ (C1). Suppose a lecture w overlaps two regions. The lecture is a contiguous arc, so it contains the whole gap between two cyclically consecutive regions X, Y and meets both; then some copy of w starts before x_e and ends after y_s, giving $Q(x_e) > y_s$ and contradicting the transition test (C2). If w meets no region it lies strictly inside one gap, so $T(x_e) \le y_s$, again contradicted (C3). Lemma 1 finishes.

Completeness. Take any valid S with blocks $B_1, \ldots, B_k$ in cyclic order. By Lemma 1 exactly one block meets lecture 1; the loop tries it as the anchor. Conditions (C1)–(C3) translate, by the argument above, into the transition and closing tests, so the DP reaches B_k and closes. The special case of a block covering the whole circle ($k = 1$) is checked directly: its validity needs only $\mathrm{N}(a) \cap \mathrm{N}(b) = \emptyset$. □

6 Complexity

Building the masks and adjacency sets costs $O(n^2)$ bit-set operations on L-bit (respectively n-bit) words. Listing blocks needs $O(n^2)$ tests of $\mathrm{N}(a) \cap \mathrm{N}(b)$. By Lemma 2 there are $m \le 2n$ blocks, hence at most $2n$ anchors. For one anchor, Q and T cost $O(n \log n)$ and the DP examines all pairs of blocks, $O(m^2) = O(n^2)$. Total

$$\boxed{T(n) = O(n^3) \text{ time}, \qquad S(n) = O(n^2) \text{ bits of space}}$$

(plus $O(nL)$ bits for the minute masks, a constant for fixed $L = 1440$). Section 7 gives an empirical check.

7 Live test cases (executed while this document compiles)

The next block is produced *during compilation*: LaTeX calls `python3 lecture_solver.py --latex-report`, which runs the solver on the built-in tests, verifies each answer independently, compares with exhaustive search, and writes the table below.

Test	Lectures (start–end)	Algorithm output	OK
T1 two overlapping lectures	L1 09:00–10:00, L2 09:30–10:30	$S = \{1, 2\}$	✓
T2 night lecture wraps midnight	L1 23:00–01:00, L2 00:30–02:00	$S = \{1, 2\}$	✓
T3 two chains around the day	L1 01:00–03:00, L2 02:00–04:00, L3 13:00–15:00, L4 14:00–16:00	$S = \{1, 2, 3, 4\}$	✓

Test	Lectures (start–end)	Algorithm output	OK
T4 four blocks, private lectures	L1 00:00–02:00, L2 01:00–03:00, L3 00:30–01:00, L4 08:00–10:00, L5 09:00–11:00, L6 08:30–09:00, L7 16:00–18:00, L8 17:00–19:00	$S = \{1, 2, 4, 6, 7, 8\}$	✓
T5 lecture touched by two blocks	L1 01:00–03:00, L2 02:00–04:00, L3 05:00–07:00, L4 06:00–08:00, L5 03:30–05:30	none	✓
T6 full-day lecture + private partner	L1 00:00–00:00, L2 10:00–11:00	$S = \{1, 2\}$	✓
T7 isolated lecture	L1 09:00–10:00, L2 09:30–10:30, L3 15:00–16:00	none	✓
T8 touching endpoints do not overlap	L1 09:00–10:00, L2 10:00–11:00	none	✓

Randomised cross-check. 400 random instances (period 24, $2 \le n \le 9$) were solved by the polynomial algorithm and by exhaustive search; 79 of them admit a valid subset. Disagreements: **0**.

All built-in tests passed.

8 Implementation (README)

Run.

```
python3 lecture_solver.py sample.txt      # solve an input file
python3 lecture_solver.py --demo          # built-in tests
python3 lecture_solver.py --stress 1000   # compare with brute force
pdflatex -shell-escape report.tex         # rebuild this PDF (runs the code)
```

Input: first line n, then n lines `HH:MM HH:MM` (start, end). The complete source (the very file executed above) follows.

```
#!/usr/bin/env python3
"""
Lecture Supervision  --  Efficient Open Domination on a Circular-Arc Graph
=========================================================================

Problem
-------
n lectures repeat every day (period L = 1440 minutes).  A lecture is a
circular arc [start, end) and may wrap past midnight.  Choose a subset S such
that EVERY lecture overlaps EXACTLY ONE lecture of S (a lecture never counts
itself).  Equivalently: S is an *efficient open dominating set* (total perfect
code) of the circular-arc "overlap" graph.

Overlap convention
------------------
Lectures are half-open arcs [start, end): a lecture ending at 13:00 and one
```

```
starting at 13:00 do NOT overlap.  start == end means a full 24-hour lecture.

Usage
-----
    python3 lecture_solver.py                  # type the lectures in interactively
    python3 lecture_solver.py input.txt        # solve a file ("-" = read stdin)
    python3 lecture_solver.py --demo           # built-in examples
    python3 lecture_solver.py --stress 500     # compare with brute force
    python3 lecture_solver.py --latex-report   # LaTeX fragment (used by report.tex)

Input format:  first line n, then n lines "HH:MM HH:MM"  (start end); "#" starts a comment.
Raw integers (minutes) are accepted as well.
"""
from __future__ import annotations

import argparse
import itertools
import random
import sys
from bisect import bisect_left

DAY = 1440  # minutes per day
INF = float("inf")


# ---------------------------------------------------------------------------
# 1.  Parsing and basic geometry
# ---------------------------------------------------------------------------
def parse_time(token: str, period: int = DAY) -> int:
    """'HH:MM' or a plain integer -> integer time in [0, period)."""
    if ":" in token:
        hh, mm = token.split(":")
        return (int(hh) * 60 + int(mm)) % period
    return int(token) % period


def fmt_time(t: int) -> str:
    return f"{t // 60:02d}:{t % 60:02d}"


def to_arcs(pairs, period=DAY):
    """[(start, end)] -> [(start, duration)], duration in 1..period."""
    arcs = []
    for s, e in pairs:
        s %= period
        d = (e - s) % period
        arcs.append((s, d if d else period))
    return arcs


def arc_mask(s: int, d: int, period: int) -> int:
    """Bit-mask of the unit cells covered by the arc (cyclic)."""
    full = (1 << period) - 1
    m = ((1 << d) - 1) << s
    return (m | (m >> period)) & full


def build_graph(arcs, period):
    """Return (cell masks, adjacency bitsets over lecture indices)."""
    n = len(arcs)
    masks = [arc_mask(s, d, period) for s, d in arcs]
    adj = [0] * n
    for i in range(n):
        for j in range(i + 1, n):
            if masks[i] & masks[j]:
                adj[i] |= 1 << j
                adj[j] |= 1 << i
    return masks, adj


# ---------------------------------------------------------------------------
# 2.  Independent checkers (used for verification only)
# ---------------------------------------------------------------------------
```

```python
def verify(pairs, chosen, period=DAY) -> bool:
    """True iff every lecture overlaps exactly one chosen lecture."""
    arcs = to_arcs(pairs, period)
    _, adj = build_graph(arcs, period)
    cmask = 0
    for c in chosen:
        cmask |= 1 << c
    return all(bin(adj[v] & cmask).count("1") == 1 for v in range(len(arcs)))


def brute_force(pairs, period=DAY):
    """Exponential reference solver (only for small n)."""
    n = len(pairs)
    arcs = to_arcs(pairs, period)
    _, adj = build_graph(arcs, period)
    for subset in range(1, 1 << n):
        if all(bin(adj[v] & subset).count("1") == 1 for v in range(n)):
            return [i for i in range(n) if subset >> i & 1]
    return None


# ---------------------------------------------------------------------------
# 3.  The polynomial algorithm
# ---------------------------------------------------------------------------
def _run_start_len(umask: int, period: int):
    """Start cell and length of the (single) run of 1-cells in umask."""
    full = (1 << period) - 1
    if umask == full:
        return None
    rot = ((umask << 1) | (umask >> (period - 1))) & full  # cell c-1 -> bit c
    start_bit = umask & ~rot
    return start_bit.bit_length() - 1, bin(umask).count("1")


def candidate_blocks(arcs, masks, adj, period):
    """
    A *block* is an overlapping pair (a, b) with N(a) & N(b) = {} .
    Every solution is a disjoint union of blocks, so only these pairs matter.
    Returns list of (a, b, union_mask, run) ; run = None if union is the circle.
    """
    n = len(arcs)
    blocks = []
    for a in range(n):
        for b in range(a + 1, n):
            if adj[a] >> b & 1 and (adj[a] & adj[b]) == 0:
                u = masks[a] | masks[b]
                blocks.append((a, b, u, _run_start_len(u, period)))
    return blocks


def solve(pairs, period=DAY):
    """
    Return a sorted list of 0-based lecture indices forming a valid subset,
    or None if none exists.   Time O(n^3), see the report.
    """
    n = len(pairs)
    if n < 2:
        return None
    L = period
    arcs = to_arcs(pairs, L)
    masks, adj = build_graph(arcs, L)
    blocks = candidate_blocks(arcs, masks, adj, L)

    # Special case: one block already wraps the whole circle.
    for a, b, _, run in blocks:
        if run is None:
            return [a, b]

    # Lecture 0 is touched by exactly one block in any solution -> anchor it.
    anchors = [k for k, blk in enumerate(blocks) if blk[2] & masks[0]]

    for x0 in anchors:
        chain = _try_anchor(x0, blocks, arcs, L)
        if chain is not None:
```

```
            return sorted(v for k in chain for v in blocks[k][:2])
    return None


def _try_anchor(x0, blocks, arcs, L):
    """Linear DP over blocks, with block x0 placed at coordinate 0."""
    s0, len0 = blocks[x0][3]

    # lecture arcs in coordinates where x0 starts at 0
    sig = [(s - s0) % L for s, _ in arcs]
    end = [sg + d for sg, (_, d) in zip(sig, arcs)]
    order = sorted(range(len(arcs)), key=lambda i: sig[i])
    sorted_sig = [sig[i] for i in order]
    pref_max = [-INF]                       # pref_max[k] = max end of first k
    for i in order:
        pref_max.append(max(pref_max[-1], end[i]))
    suf_min = [INF] * (len(arcs) + 1)       # suf_min[k] = min end from k on
    for k in range(len(arcs) - 1, -1, -1):
        suf_min[k] = min(suf_min[k + 1], end[order[k]])
    wrap_max = max(e - L for e in end)      # copies shifted by -L

    def Q(p):   # furthest right end of a lecture that starts before p
        return max(pref_max[bisect_left(sorted_sig, p)], wrap_max)

    def T(p):   # earliest end of a lecture that starts at or after p
        return suf_min[bisect_left(sorted_sig, p)]

    # nodes: (start, end, block-id); node 0 is the anchor itself
    nodes = [(0, len0, x0)]
    others = []
    for k, blk in enumerate(blocks):
        if k == x0:
            continue
        s_k, l_k = blk[3]
        ys = (s_k - s0) % L
        if ys >= len0 and ys + l_k <= L:   # disjoint from the anchor
            others.append((ys, ys + l_k, k))
    nodes += sorted(others)

    m = len(nodes)
    reach = [False] * m
    parent = [-1] * m
    reach[0] = True
    Qe = [Q(nd[1]) for nd in nodes]
    Te = [T(nd[1]) for nd in nodes]

    def chain_to(j):
        out = []
        while j != -1:
            out.append(nodes[j][2])
            j = parent[j]
        return out[::-1]

    # k = 1 : the anchor alone covers everything
    if L < Te[0]:
        return [x0]

    for j in range(1, m):
        ys = nodes[j][0]
        for i in range(j):
            if reach[i] and nodes[i][1] <= ys and Qe[i] <= ys < Te[i]:
                reach[j], parent[j] = True, i
                break
        # closing edge  j -> anchor shifted by L   (needs k >= 2)
        if reach[j] and Qe[j] <= L < Te[j]:
            return chain_to(j)
    return None


# ---------------------------------------------------------------------------
# 4.  Reporting helpers
# ---------------------------------------------------------------------------
def parse_tokens(tok, period=DAY):
    """Token list [n, s1, e1, s2, e2, ...] -> list of (start, end)."""
```

```
    if not tok:
        raise ValueError("empty input")
    n = int(tok[0])
    if n < 0 or len(tok) != 1 + 2 * n:
        raise ValueError(f"expected n and then {n} start/end pairs "
                         f"({1 + 2 * n} values), got {len(tok)} values")
    return [(parse_time(tok[1 + 2 * i], period), parse_time(tok[2 + 2 * i], period))
            for i in range(n)]


def read_file(path, period=DAY):
    """Read a file, or standard input when path is '-'. '#' starts a comment."""
    fh = sys.stdin if path == "-" else open(path)
    with fh:
        text = "\n".join(line.split("#")[0] for line in fh)
    return parse_tokens(text.split(), period)


def interactive(period=DAY):
    """Ask the user to type the lectures one by one."""
    n = int(input("Number of lectures n: "))
    pairs = []
    for i in range(1, n + 1):
        while True:
            try:
                s, e = input(f"Lecture {i} - start end (HH:MM HH:MM): ").split()
                pairs.append((parse_time(s, period), parse_time(e, period)))
                break
            except ValueError:
                print("  please enter two times such as 09:30 11:00")
    return pairs


def hm(s, e):
    return (parse_time(s), parse_time(e))


# ---------------------------------------------------------------------------
# 5.  Built-in test cases  (name, lectures, expected: solvable?)
# ---------------------------------------------------------------------------
TESTS = [
    ("T1  two overlapping lectures",
     [hm("09:00", "10:00"), hm("09:30", "10:30")], True),
    ("T2  night lecture wraps midnight",
     [hm("23:00", "01:00"), hm("00:30", "02:00")], True),
    ("T3  two chains around the day",
     [hm("01:00", "03:00"), hm("02:00", "04:00"),
      hm("13:00", "15:00"), hm("14:00", "16:00")], True),
    ("T4  four blocks, private lectures",
     [hm("00:00", "02:00"), hm("01:00", "03:00"), hm("00:30", "01:00"),
      hm("08:00", "10:00"), hm("09:00", "11:00"), hm("08:30", "09:00"),
      hm("16:00", "18:00"), hm("17:00", "19:00")], True),
    ("T5  lecture touched by two blocks",
     [hm("01:00", "03:00"), hm("02:00", "04:00"),
      hm("05:00", "07:00"), hm("06:00", "08:00"),
      hm("03:30", "05:30")], False),
    ("T6  full-day lecture + private partner",
     [hm("00:00", "00:00"), hm("10:00", "11:00")], True),
    ("T7  isolated lecture",
     [hm("09:00", "10:00"), hm("09:30", "10:30"), hm("15:00", "16:00")], False),
    ("T8  touching endpoints do not overlap",
     [hm("09:00", "10:00"), hm("10:00", "11:00")], False),
]


def random_instance(rng, n, period):
    return [(rng.randrange(period), rng.randrange(period)) for _ in range(n)]


def stress(trials, seed=1, max_n=9, period=24):
    """Compare solve() with brute_force(); return statistics."""
    rng = random.Random(seed)
    solvable = bad = 0
    for _ in range(trials):
```

```
        n = rng.randint(2, max_n)
        inst = random_instance(rng, n, period)
        got = solve(inst, period)
        ref = brute_force(inst, period)
        if (got is None) != (ref is None) or (got is not None and not verify(inst, got,
            period)):
            bad += 1
        solvable += ref is not None
    return trials, solvable, bad


# ---------------------------------------------------------------------------
# 6.  LaTeX fragment  (consumed by report.tex through \input)
# ---------------------------------------------------------------------------
def _tex(s):
    return s.replace("&", r"\&").replace("_", r"\_").replace("#", r"\#")


def latex_report():
    out = [r"\begin{longtable}{p{4.4cm}p{3.2cm}p{2.6cm}c}",
           r"\toprule Test & Lectures (start--end) & Algorithm output & OK \\ \midrule \
               endhead"]
    all_ok = True
    for name, pairs, expect in TESTS:
        got = solve(pairs)
        ref = brute_force(pairs)
        ok = ((got is None) == (ref is None) == (not expect)) and \
             (got is None or verify(pairs, got))
        all_ok &= ok
        lec = ", ".join(f"L{i + 1}\\,{fmt_time(s)}--{fmt_time(e)}"
                        for i, (s, e) in enumerate(pairs))
        res = "none" if got is None else "$S=\\{" + ",".join(str(c + 1) for c in got) + "
            \\}$"
        mark = "\\checkmark" if ok else "\\texttimes"
        out.append(f"{_tex(name)} & {{\\footnotesize {lec}}} & {res} & {mark} \\\\")
    out.append(r"\bottomrule\end{longtable}")
    trials, sol, bad = stress(400)
    out.append(
        f"\\paragraph{{Randomised cross-check.}} {trials} random instances "
        f"(period 24, $2\\le n\\le 9$) were solved by the polynomial algorithm and by "
        f"exhaustive search; {sol} of them admit a valid subset. "
        f"Disagreements: \\textbf{{{bad}}}.")
    out.append("\\par\\medskip\\noindent\\textbf{All built-in tests passed.}"
               if all_ok and bad == 0 else
               "\\par\\medskip\\noindent\\textbf{WARNING: a test failed.}")
    return "\n".join(out)


# ---------------------------------------------------------------------------
# 7.  Command line
# ---------------------------------------------------------------------------
def report(pairs, name="input"):
    sol = solve(pairs)
    print(f"--- {name} (n = {len(pairs)}) ---")
    if sol is None:
        print("No valid subset exists.")
        return
    print("Selected lectures (1-indexed):", [c + 1 for c in sol])
    cmask = sum(1 << c for c in sol)
    arcs = to_arcs(pairs)
    _, adj = build_graph(arcs, DAY)
    for v, (s, d) in enumerate(arcs):
        sup = [u + 1 for u in range(len(arcs)) if (adj[v] >> u & 1) and (cmask >> u & 1)]
        print(f"  lecture {v + 1:>3} {fmt_time(s)}-{fmt_time((s + d) % DAY)}"
              f"  supervised by {sup}")
    print("Independent verification:", "PASSED" if verify(pairs, sol) else "FAILED")


def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                 formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("file", nargs="?")
    ap.add_argument("--demo", action="store_true")
    ap.add_argument("--stress", type=int, metavar="N")
```

```
    ap.add_argument("--latex-report", action="store_true")
    a = ap.parse_args()

    if a.latex_report:
        print(latex_report())
    elif a.stress:
        t, s, b = stress(a.stress)
        print(f"{t} random instances, {s} solvable, {b} disagreements with brute force")
    elif a.demo:
        for name, pairs, _ in TESTS:
            report(pairs, name)
    elif a.file:
        try:
            report(read_file(a.file), a.file)
        except (ValueError, OSError) as err:
            sys.exit(f"Input error: {err}")
    else:                          # no arguments: type the lectures in
        report(interactive(), "typed input")


if __name__ == "__main__":
    main()
```